

Duale Hochschule Baden-Württemberg Mannheim

Studienarbeit

Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III

Studiengang Informatik

Studienrichtung Angewandte Informatik

Verfasser:	Ertan Ertas und Christian Strerath
Matrikelnummer:	9390224, 4965013
Kurs:	TINF23AI2
Studiengangsleiter:	Prof. Dr. Holger Hofmann
Betreuer:	Prof. Dr. Karl Stroetmann
Bearbeitungszeitraum:	15.10.2025 - 14.04.2026

Erklärung

Wir versichern hiermit, dass wir unsere Studienarbeit mit dem Thema: „Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Mannheim, 5. März 2026

Ertan Ertas

Kaiserslautern, 5. März 2026

Christian Strerath

Kurzfassung (Abstract)

Auf Deutsch

In English

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung	3
1.3	Methodik und Abgrenzung	3
2	Grundlagen der Typisierung	5
2.1	Python und die Grenzen dynamischer Typisierung	5
2.2	TypeScript: Statische Sicherheit zur Compile-Zeit	10
2.3	Strukturelles Typing: Verträge statt Vererbung	10
2.4	Generics: Typen als Parameter	10
2.5	Literal Types: Werte als Typen	10
2.6	Type Narrowing: Typ-Prüfung im Kontrollfluss	10
2.7	Gegenüberstellung: Paradigmen im Vergleich	10
3	Literaturverzeichnis	11
A	Anhang	12
A.1	Verzeichnis der portierten Lehrnotebooks	12

Abbildungsverzeichnis

Tabellenverzeichnis

1	Übersicht der übersetzten Notebooks im Repository	12
---	---	----

List of Listings

1	Mengen in Python: Das erwartete Verhalten	2
2	Mengen in TypeScript: Das fehlerhafte Verhalten bei Tupeln	2
3	Dynamische Typen in Python	5
4	Duck Typing in Python	6
5	Daten- und Methoden-Attribute in Python	7
6	Type Hints in Python haben zur Laufzeit keine Auswirkung	8
7	Typ-Verlust durch dynamische Code-Ausführung in Jupyter-Notebooks	9

Glossar

Repository Ein zentraler, oft versionierter Speicherort für Quellcode und digitale Dokumente, der es ermöglicht, Änderungen am Code historisch nachzuvollziehen und gemeinsam an Projekten zu arbeiten. 4

1 Einleitung

Die Theoretische Informatik bildet ein wichtiges Fundament im Informatikstudium. Themen wie Automaten oder formale Sprachen können anfangs jedoch sehr abstrakt wirken. Um diese theoretischen Konzepte greifbarer zu machen, kommen in der Vorlesung „Theoretische Informatik III“ interaktive Notebooks zum Einsatz. Dort wird erklärender Text direkt mit ausführbarem Programmcode gemischt. Bislang basieren diese Notebooks auf der Programmiersprache Python. Studierende können so theoretische Algorithmen direkt ausprobieren, verändern und live nachvollziehen.

Diese Arbeit befasst sich mit der Übertragung dieser Lehrinhalte in die Programmiersprache TypeScript. Der Fokus liegt dabei auf einer Portierung, welche nicht nur die Syntax übersetzt, sondern die Konzepte der formalen Sprachen durch verbindliche Datenstrukturen explizit modelliert.

1.1 Problemstellung

Ausgangspunkt dieser Arbeit sind interaktive Python-Notebooks, die Konzepte der theoretischen Informatik demonstrieren. Werden diese Lehrmaterialien in das TypeScript-Ökosystem übertragen, reicht es jedoch nicht aus, lediglich die Syntax der Sprache zu übersetzen. Es entstehen fundamentale technische Hürden, da beide Sprachen Daten auf unterschiedliche Weise verarbeiten. Folgende zwei Problemfelder stehen dabei im Fokus:

Zu späte Fehlererkennung bei verschachtelten Daten

In Python haben Variablen keine festen Typen. Das führt oft dazu, dass Programmierfehler erst auffallen, wenn der Code ausgeführt wird und plötzlich abstürzt. Zwar gibt es externe Werkzeuge, um Datentypen in Python nachträglich zu prüfen, diese sind jedoch nicht nativ in die Sprache integriert und stoßen gerade in der interaktiven Umgebung der Notebooks oft an ihre Grenzen.

In der Vorlesung arbeiten wir ständig mit tief verschachtelten Objekten, beispielsweise Datenstrukturen, bei denen eine Liste wieder viele weitere Objekte und Listen enthält. Fehlt einem dieser verschachtelten Elemente versehentlich eine wichtige Eigenschaft, merkt Python das erst, wenn der Code im Hintergrund genau dieses fehlerhafte Element aufruft. Wir benötigen stattdessen eine Sprache, die uns schon während des Tippens garantiert, dass unsere Datenstrukturen über alle Ebenen hinweg fehlerfrei und vollständig aufgebaut sind.

Das unerwartete Verhalten von Mengen (Sets)

Das gravierendste Hindernis bei der Übersetzung zeigt sich jedoch bei der Arbeit mit Mengen (Sets). In der theoretischen Informatik ist es unumgänglich, berechnete Zustände in Mengen zu speichern, um Duplikate zu vermeiden oder später zu prüfen, ob ein Zustand bereits erreicht wurde.

In Python funktioniert das exakt so, wie man es intuitiv erwartet. Fügen wir ein Tupel in eine Menge ein und fragen danach ab, ob ein Tupel mit denselben Zahlen enthalten ist, liefert Python das erwartete Ergebnis:

```
1  menge = set()
2  menge.add((1, 2))
3
4  # Die Menge enthält das Tupel sichtbar
5  print(menge)           # Ausgabe: {(1, 2)}
6
7  # Die Abfrage liefert das erwartete Ergebnis
8  print((1, 2) in menge) # Ausgabe: True
```

Listing 1: Mengen in Python: Das erwartete Verhalten

Ein naiver Übersetzungsversuch nach TypeScript führt hier jedoch zu einem fundamentalen Bruch in der Logik. Zwar erlaubt uns die Sprache, im Code ganz genau festzulegen, dass unsere Menge ausschließlich Tupel aus exakt zwei Zahlen aufnehmen darf. Führen wir diesen Code jedoch aus und fragen das zuvor hinzugefügte Element ab, erhalten wir plötzlich ein falsches Ergebnis:

```
1  // Wir definieren den Typ Tuple, der zwei Zahlen akzeptiert.
2  type Tuple = [number, number];
3
4  // Die Menge soll nur diese Tupel akzeptieren
5  const menge = new Set<Tuple>();
6  menge.add([1, 2]);
7
8  // Die Menge enthält das Element sichtbar
9  console.log(menge);           // Ausgabe: Set(1) { [ 1, 2 ] }
10
11 // Die Abfrage liefert ein ungewöhnliches Ergebnis
12 console.log(menge.has([1, 2])); // Ausgabe: false
```

Listing 2: Mengen in TypeScript: Das fehlerhafte Verhalten bei Tupeln

Obwohl die Konsolenausgabe beweist, dass das Element `[1, 2]` offensichtlich in die Menge eingefügt wurde, behauptet TypeScript's Standard-Set in der

anschließenden Abfrage, es sei nicht vorhanden. Dieser zunächst unerklärliche Widerspruch führt bei einer direkten Portierung dazu, dass Logik-Abfragen fehlschlagen, Endlosschleifen entstehen und Algorithmen falsche Ergebnisse liefern.

1.2 Zielsetzung

Das übergeordnete Ziel dieser Arbeit ist es, die Lehrinhalte der Vorlesung „Theoretische Informatik III“ vollständig nach TypeScript zu übersetzen. Die neue Codebasis soll die bisherigen Python-Notebooks ersetzen und dabei die strikten Typvorgaben der neuen Sprache nutzen, um Fehler schon während des Schreibens zu verhindern.

Daraus leiten sich folgende konkrete Teilziele ab:

1. **Strukturierte Neuimplementierung:** Die Algorithmen der Vorlesung sollen vollständig in TypeScript portiert werden. Der fehleranfällige Verzicht auf feste Typen in Python wird dabei durch verbindliche Datenstrukturen und strikte Typvorgaben ersetzt.
2. **Lösung des Mengen-Problems:** Um die Algorithmen fehlerfrei auszuführen, muss das fehlerhafte Verhalten von TypeScripts Standard-Sets umgangen werden. Das zentrale technische Ziel ist daher die Entwicklung einer eigenen Datenstruktur, die sich beim Vergleichen von Inhalten exakt wie ein Python-Set verhält und auch tief verschachtelte Objekte verlässlich erkennt.
3. **Einheitliche Ausführungsumgebung:** Da die Notebooks lokal auf den Rechnern der Studierenden ausgeführt werden, muss eine standardisierte Umgebung geschaffen werden, die unabhängig vom Betriebssystem identisch funktioniert. Ein wichtiges technisches Teilziel ist es dabei, die Hintergrundprozesse der Notebooks so anzupassen, dass TypeScripts strenge Prüfregeln auch bei der interaktiven, schrittweisen Ausführung zuverlässig greifen. Der Einrichtungsaufwand soll durch eine klare Anleitung minimal gehalten werden.

1.3 Methodik und Abgrenzung

Die methodische Durchführung dieser Arbeit folgt einem fokussierten Ansatz. Der Umfang der Portierung beschränkt sich auf die nummerierten Lehrnotebooks der Vorlesung sowie deren unmittelbare technische Abhängigkeiten. Übungsnotebooks, die primär dem unbetreuten Selbststudium dienen, sind explizit vom Umfang dieser Arbeit ausgeklammert. Die genaue Auswahl der zu übersetzenden

Artefakte erfolgte in Abstimmung mit dem Betreuer und ist im Anhang (siehe Abschnitt A.1) tabellarisch aufgeführt.

Ein zentraler methodischer Aspekt betrifft die didaktische Aufbereitung: Die begleitenden Erklärtexpte der Notebooks wurden nicht isoliert übersetzt, sondern inhaltlich an die neuen Strukturen von TypeScript angepasst. Dabei wurde bewusst darauf verzichtet, die neuen Konzepte kontinuierlich mit der alten Python-Implementierung zu vergleichen. Die Notebooks sollen als eigenständiges Lehrmaterial funktionieren, das die theoretischen Konzepte nativ und in sich geschlossen vermittelt.

Auf technischer Ebene war die Minimierung von Systemabhängigkeiten ein wesentliches Entwurfsziel. Insbesondere für die grafische Visualisierung von Automaten und Graphen wurde darauf geachtet, keine komplexen lokalen Installationen von externer Software vorauszusetzen. Stattdessen kommen integrierbare, plattformunabhängige Lösungen zum Einsatz, um die Ausführung auf verschiedenen Betriebssystemen zu garantieren und den Einstieg für die Studierenden so barrierefrei wie möglich zu gestalten.

Um die Ergebnisse dieser Arbeit nachvollziehbar und frei zugänglich zu machen, wird der gesamte entwickelte Quellcode in einem öffentlichen Repository bereitgestellt. Die Implementierung ist unter der URL <https://github.com/cstreich/fomal-languages-typescript> im Verzeichnis TypeScript abrufbar.

2 Grundlagen der Typisierung

Was ist ein Typ? Im Arbeitsspeicher eines Computers sind alle Daten letztlich nur endlose Reihen von Nullen und Einsen. Erst das Konzept eines „Typs“ gibt dieser rohen Bitfolge eine Bedeutung. Der Typ entscheidet, ob eine bestimmte Bitfolge als ganze Zahl, als Buchstabe oder als komplexes Objekt (wie etwa ein Suchbaum) interpretiert werden soll. Gleichzeitig dienen Typen als Leitplanken für uns Entwickler: Sie verhindern beispielsweise, dass wir versehentlich Text mit Zahlen multiplizieren.

Im Folgenden betrachten wir, wie unterschiedlich Programmiersprachen diese Leitplanken umsetzen. Dabei vergleichen wir unsere Ausgangssprache Python mit unserer Zielsprache TypeScript und zeigen auf, warum bestimmte Paradigmen für komplexe Projekte besser geeignet sind.

2.1 Python und die Grenzen dynamischer Typisierung

In Python wird der Typ einer Variable nicht beim Schreiben des Codes festgelegt, sondern erst dann bestimmt, wenn das Programm tatsächlich ausgeführt wird. Das folgende Beispiel zeigt, was das in der Praxis bedeutet:

```
1 x = 42           # x ist jetzt eine Zahl
2 x = "Hello"      # x ist jetzt ein Text, kein Fehler, kein Einwand
3 x = [1, 2, 3]    # x ist jetzt eine Liste, Python akzeptiert das stillschweigend
4
5 print(x + 1)     # Erst hier stürzt das Programm ab: TypeError
```

Listing 3: Dynamische Typen in Python

Python führt standardmäßig keine Vorab-Prüfung durch. Das Programm startet, arbeitet die Zeilen nacheinander ab und bemerkt den Fehler erst in dem Moment, in dem er tatsächlich auftritt.

Dieses Verhalten hat einen technischen Grund, der in der Ausführungsart der Sprache liegt. Python arbeitet mit einem sogenannten *Interpreter*. Ein Interpreter ist ein Programm, das den Quellcode einliest und ihn direkt, Schritt für Schritt, ausführt. Er analysiert den Code nicht vorab im Gesamten. Wie im obigen Beispiel in Zeile 5 zu sehen ist, bemerkt der Interpreter den fatalen Typfehler bei der Ausführung von `print(x + 1)` erst in dem exakten Moment, in dem er diese Zeile erreicht. Vorher „weiß“ er schlicht nichts von dem Problem.

Der TypeError: Wenn die Operation nicht zum Typ passt

Wenn der Interpreter in Zeile 5 schließlich ankommt, prüft er den aktuellen Zustand: `x` ist eine Liste und die Operation verlangt die Addition mit einer ganzen Zahl (`int`). Diese Operation ist für diese Kombination von Datentypen schlicht nicht definiert, weshalb der Interpreter die Ausführung strikt verweigert. Das Resultat ist ein `TypeError`.

Im Gegensatz dazu nutzen statisch typisierte Sprachen (wie unsere Zielsprache TypeScript) einen *Compiler*. Ein Compiler ist ein Übersetzungsprogramm, das als vorgeschalteter Wächter agiert. Er liest und analysiert den *gesamten* Quellcode vor der Ausführung auf logische Widersprüche und Typfehler. Erst wenn der Code fehlerfrei ist, wird er übersetzt und darf ausgeführt werden. Ein Programm mit dem Typfehler aus Zeile 5 würde bei einem Compiler-Ansatz mit einer Fehlermeldung abgelehnt werden und könnte gar nicht erst starten.

Duck Typing

Weil Python Typen so locker handhabt, folgt die Sprache einem Prinzip, das als *Duck Typing* bekannt ist. Anstatt den genauen Typ eines Objekts zu prüfen, wird einfach versucht, die benötigte Methode aufzurufen [1]. Der Fokus liegt also nicht darauf, *was* ein Objekt ist, sondern nur darauf, *was* es kann. Der Name leitet sich von der bekannten Redewendung ab: „Wenn es wie eine Ente läuft und wie eine Ente quakt, dann muss es eine Ente sein.“

```
1 def make_sound(animal):
2     # Python prüft nicht, was "animal" ist, nur ob es .sound() aufrufen kann
3     animal.sound()
4
5 class Duck:
6     def sound(self):
7         print("Quack!")
8
9 class Dog:
10    def sound(self):
11        print("Woof!")
12
13 class Rock:
14    pass # hat keine sound()-Methode
15
16 make_sound(Duck()) # Ausgabe: Quack!
17 make_sound(Dog()) # Ausgabe: Woof!
18 make_sound(Rock()) # Erst hier stürzt das Programm ab: AttributeError
```

Listing 4: Duck Typing in Python

Die Funktion `make_sound` akzeptiert jeden beliebigen Wert. Ob der übergebene Wert tatsächlich eine Methode `sound()` besitzt, stellt sich erst bei der Ausführung heraus. Für kleine Skripte ist das flexibel, in einer größeren Codebasis, wie den Vorlesungsnotebooks, wird dieses Verhalten jedoch zum Problem: Ein fehlerhafter Aufruf kann tief im Inneren eines Algorithmus verborgen sein und erst sehr spät zur Laufzeit einen Absturz verursachen.

Was ist eigentlich ein Attribut?

Um den daraus resultierenden Fehler zu verstehen, müssen wir kurz klären, was Python unter einem *Attribut* versteht. In Python ist fast alles ein Objekt. Man kann sich ein Objekt stark vereinfacht als einen Container vorstellen, der Variablen und Funktionen bündelt. Alles, was an ein solches Objekt angeheftet ist und über die Punkt-Notation (`objekt.name`) abgerufen wird, nennt man Attribut.

```
1 class Player:
2     def __init__(self):
3         # 'health' ist ein Daten-Attribut (eine Variable, die zum Objekt gehört)
4         self.health = 100
5
6         # 'jump' ist ein Methoden-Attribut (eine Funktion, die zum Objekt gehört)
7         def jump(self):
8             print("Jump!")
9
10 p = Player()
11 print(p.health) # Greift auf das Daten-Attribut 'health' zu
12 p.jump()       # Greift auf das Methoden-Attribut 'jump' zu und führt es aus
```

Listing 5: Daten- und Methoden-Attribute in Python

Wenn ein Attribut – wie `jump` im obigen Beispiel – ausführbar ist, bezeichnen wir es im Entwickler-Alltag meist als *Methode*. Für den Python-Interpreter ist es technisch gesehen jedoch zunächst nur ein Name (ein Attribut), nach dem er am Objekt sucht, sobald der Punkt aufgerufen wird.

Der `AttributeError`: Wenn das Attribut fehlt

Genau hier kommt das späte Scheitern des Duck Typings ins Spiel, welches sich in der Regel durch einen `AttributeError` äußert. Wenn wir in Zeile 18 unseres Enten-Beispiels `make_sound(Rock())` aufrufen, sucht der Interpreter zur Laufzeit am übergebenen Stein-Objekt nach einem Attribut namens `sound`. Da die Klasse `Rock` dieses Attribut nicht definiert hat, schlägt die Suche fehl. Während der `TypeError` also aussagt „Ich kann diese Operation mit diesem Typ nicht ausführen“, bedeutet der `AttributeError` schlicht: „Das Objekt besitzt das geforderte Attribut überhaupt

nicht.“

Type Hints als freiwillige Dokumentation

Um der fehlenden Typsicherheit entgegenzuwirken, führte Python sogenannte *Type Hints* ein. Entwickler können Variablen, Parameter und Rückgabewerte mit *Typ-Annotationen* versehen – das sind syntaktische Markierungen wie `: int` oder `-> str`, die den gewünschten Datentyp im Code sichtbar machen. Wichtig ist hierbei: Python selbst ignoriert diese Hinweise zur Ausführungszeit vollständig. Sie dienen nicht dem Interpreter, sondern primär dem Programmierer als Dokumentation sowie externen Werkzeugen als Metadaten.

```
1 def add(a: int, b: int) -> int:
2     return a + b
3
4 # Python führt dies fehlerfrei aus (String-Konkatenation statt Addition)
5 print(add("Hello", "World")) # Ausgabe: HelloWorld
```

Listing 6: Type Hints in Python haben zur Laufzeit keine Auswirkung

Was ist ein Linter?

Da Python die Typ-Annotationen zur Laufzeit schlicht überliest, muss ein externes Werkzeug die eigentliche Überprüfung übernehmen. Ein solches Werkzeug nennt man *Linter* (oder spezifischer: *Static Type Checker*). Ein Linter ist ein separates Analyseprogramm. Er nimmt den Quellcode, liest ihn durch und sucht nach formalen oder logischen Fehlern, wie etwa Typ-Konflikten. Das Entscheidende dabei ist: Ein Linter führt den Code *nicht* aus. Er führt eine rein statische Textanalyse durch. In der Python-Welt hat sich für die Typprüfung der Linter *mypy* als Standard etabliert. Er analysiert klassische Python-Dateien (`.py`) und warnt den Entwickler vor Fehlern, noch bevor das Programm überhaupt gestartet wird.

Die Herausforderung: Jupyter-Notebooks und nb-mypy

Die Vorlesungsmaterialien bestehen jedoch nicht aus einfachen Textdateien, sondern aus interaktiven *Jupyter-Notebooks* (`.ipynb`). In einem Notebook schreibt man den Code nicht in eine einzige große Textdatei, sondern zerteilt ihn in kleine Blöcke, die sogenannten *Zellen*.

Wenn man eine Zelle ausführt, wird nur dieser spezifische Code-Schnipsel an einen im Hintergrund laufenden Python-Prozess geschickt. Dieser Prozess merkt sich alle bisher definierten Variablen und Funktionen im Arbeitsspeicher. Das Standard-Werkzeug *mypy* ist jedoch für statische Dateien konzipiert. Es schaut nur auf den reinen Text und hat keinen Zugriff auf diesen dynamisch wachsenden

Zustand im Hintergrundspeicher.

Um dennoch eine statische Typanalyse in den Notebooks zu ermöglichen, wird im Vorlesungsprojekt die Erweiterung *nb-mypy* eingesetzt. Sie klinkt sich in die Notebook-Umgebung ein und versucht, den Code einer Zelle durch *mypy* zu prüfen, kurz bevor er an den Hintergrundprozess gesendet wird.

Der Bruch an der Dateigrenze: Das %run-Problem

Doch genau an der Schnittstelle zwischen mehreren Notebooks bricht diese Architektur zusammen. Die Vorlesungsunterlagen trennen häufig die Implementierung eines Algorithmus und dessen Tests in zwei separate Notebooks. Um den Algorithmus im Test-Notebook verfügbar zu machen, wird der *Magic Command* `%run` genutzt. Magic Commands sind spezielle Jupyter-Befehle (erkennbar am `%`), die den Hintergrundprozess anweisen, Sonderaufgaben zu erledigen. Im Fall von `%run` wird ein externes Notebook dynamisch ausgeführt, wodurch all dessen Variablen und Funktionen in den aktuellen Speicher geladen werden.

```
1 # --- Zelle im Notebook: algorithm.ipynb ---
2 def calculate_state(start_id: int) -> str:
3     return f"State_{start_id}"
4
5 # --- Zelle im Notebook: test.ipynb ---
6 # Ein Magic Command führt das externe Notebook dynamisch aus
7 %run algorithm.ipynb
8
9 # nb-mypy verliert an dieser Grenze den Typ-Kontext.
10 # Es weiß nicht, was "calculate_state" ist oder welche Signatur es hat.
11 result = calculate_state(42)
```

Listing 7: Typ-Verlust durch dynamische Code-Ausführung in Jupyter-Notebooks

Da *nb-mypy* lediglich den statischen Text der aktuellen Zelle analysiert, versteht es die Auswirkungen des dynamischen `%run`-Befehls nicht. Die Typinformationen aus dem Algorithmus-Notebook werden für den Linter nicht in das Test-Notebook übertragen.

Als Konsequenz meldet der Linter im Test-Notebook unzählige falsche Fehler, da ihm der Typ-Kontext fehlt. Um das System nutzbar zu halten, muss *nb-mypy* über Befehle wie `%nb_mypy unload` temporär komplett deaktiviert oder mit speziellen Kommentaren (`# type: ignore`) zum Schweigen gebracht werden.

Ein Typsystem, das an Dateigrenzen seine Informationen verliert und vom Entwickler händisch abgeschaltet werden muss, verfehlt seinen Zweck der verlässlichen Fehlervermeidung komplett. Es wird vom Hilfsmittel zum frustrierenden Hindernis. Diese fundamentale Inkompatibilität zwischen einer hochdynamischen

Ausführungsumgebung und einem nachträglich aufgesetzten Type Checker zeigt deutlich, warum für dieses Projekt ein kompletter Paradigmenwechsel nötig war. Wir benötigen eine Sprache, die Module, Importe und Typsicherheit nicht als externe Pflaster behandelt, sondern nativ in ihrem Fundament verankert hat. Dies führt uns im folgenden Abschnitt direkt zu unserer Zielsprache: TypeScript.

2.2 TypeScript: Statische Sicherheit zur Compile-Zeit

2.3 Strukturelles Typing: Verträge statt Vererbung

2.4 Generics: Typen als Parameter

2.5 Literal Types: Werte als Typen

2.6 Type Narrowing: Typ-Prüfung im Kontrollfluss

2.7 Gegenüberstellung: Paradigmen im Vergleich

3 Literaturverzeichnis

- [1] *Glossary*. Python Software Foundation. 2026. URL: <https://docs.python.org/3/glossary.html#term-duck-typing> (besucht am 05.03.2026).

A Anhang

A.1 Verzeichnis der portierten Lehrnotebooks

Die folgende Tabelle listet alle Jupyter-Notebooks auf, die im Rahmen dieser Arbeit selektiert und von Python nach TypeScript portiert wurden. Die Gliederung folgt der Kapitelstruktur der Originalvorlesung „Theoretische Informatik III“.

Tabelle 1: Übersicht der übersetzten Notebooks im Repository

Original-Kapitel	Notebook-Dateiname
Chapter 03	01-Ply-Scanning-Example.ipynb 02-Regex-Tutorial.ipynb 03-Exam-Evaluation.ipynb 04-Html2Text.ipynb
Chapter 04 & 05	01-NFA-2-DFA.ipynb 02-Test-NFA-2-DFA.ipynb 03-Regex-2-NFA.ipynb 04-Test-Regex-2-NFA.ipynb 05-DFA-2-RegExp.ipynb 06-Test-DFA-2-RegExp.ipynb 07-Minimize.ipynb 08-Test-Minimize.ipynb 09-Equivalence.ipynb 10-Test-Equivalence.ipynb
Chapter 06	01-Top-Down-Parser.ipynb 02-EBNF-Parser.ipynb
Chapter 07	01-Calculator.ipynb 02-Differentiator.ipynb 03-Interpreter.ipynb
Chapter 10	01-Conflicts.ipynb 02-Conflicts-Resolved.ipynb

Quelle: Eigene Darstellung.